# Outbox Pattern - Cheat Sheet

The Outbox pattern solves a common problem in distributed systems: how do you reliably publish an event after a database write, without using a distributed transaction?

The trick: write the business data and the event to the same database in one local transaction. A separate process publishes the event from the outbox table to the message broker. Either both happen or neither does.

Whenever you've written code like "save to DB, then publish to Kafka", you've probably been writing bugs. The outbox fixes that.

## The dual-write problem

A common workflow: save a domain object, then publish an event so other services react.

```
public void placeOrder(Order order) {
    repository.save(order);                       // 1. write to DB
    eventBus.publish(new OrderPlaced(order.id())); // 2. publish event
}
```

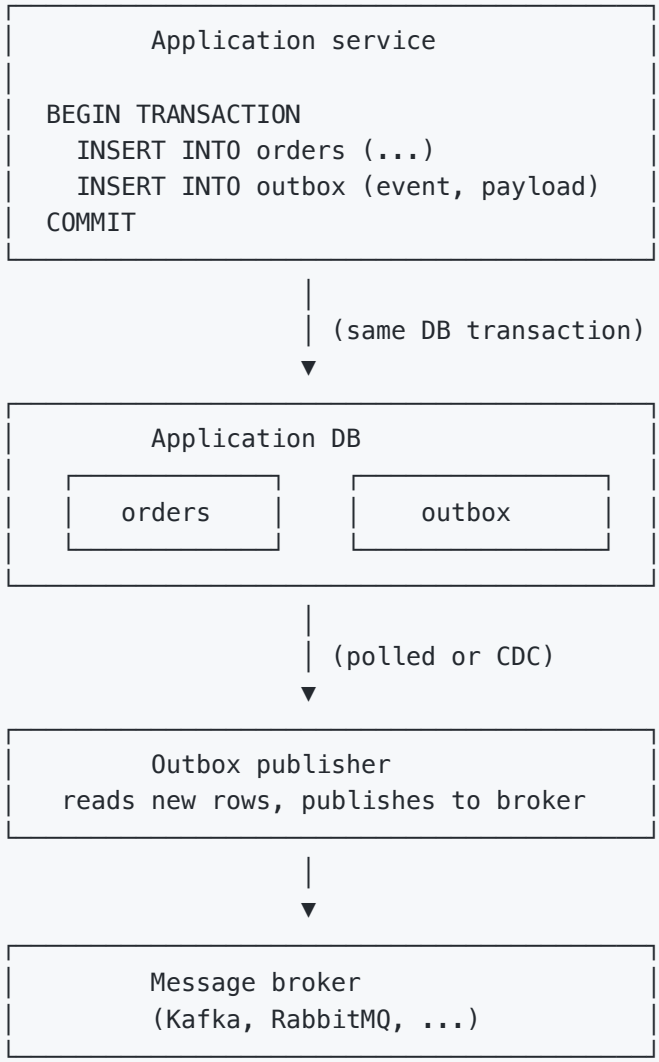
Three failure modes break this:

1. DB write fails. No event published. Caller sees an error. Consistent but failed.
2. DB write succeeds, broker is down. Order is saved, event is lost. Other services never react. Silent inconsistency.
3. DB write succeeds, event publishes, then the process crashes before returning. Caller retries. Order saved twice, event published twice. Duplicate side effects.

You can't make 2 systems agree without a distributed transaction (2PC, XA). And 2PC is expensive, blocking, and rarely supported across DBs and message brokers.

The outbox dodges 2PC by funneling both writes through one resource: the database.

## How the outbox works




The application writes both rows in one transaction. Either both land, or neither does. The publisher then carries the event from the database to the broker.

If the publisher crashes between reading and publishing, the event still sits in the outbox. It gets picked up on the next pass.

This gives you at-least-once delivery. Consumers must be idempotent.

## Outbox table schema

A typical schema:

```sql
CREATE TABLE outbox (
    id           BIGSERIAL    PRIMARY KEY,
    aggregate    VARCHAR(50)  NOT NULL,      -- e.g., 'Order', 'User'
    aggregate_id VARCHAR(100) NOT NULL,      -- the entity's ID
    event_type   VARCHAR(100) NOT NULL,      -- e.g., 'OrderPlaced'
    payload      JSONB        NOT NULL,      -- the event body
    headers      JSONB,                      -- correlation ID, tracing, etc.
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    published_at TIMESTAMPTZ                 -- null until publisher succeeds
);

CREATE INDEX idx_outbox_unpublished
    ON outbox (created_at)
    WHERE published_at IS NULL;
```

Key columns:

- `id`: monotonically increasing. Provides natural ordering.
- `aggregate` + `aggregate_id`: identifies the entity the event is about. Often used as the partition key in Kafka.
- `event_type`: what happened.
- `payload`: event data. JSON or a serialized protobuf.
- `headers`: correlation ID, trace context, schema version.
- `published_at`: set once the publisher confirms delivery. Helps with cleanup.

## Publishing mechanisms

Two main ways to get events out of the outbox.

### Polling publisher

A background job repeatedly reads unpublished rows and sends them to the broker.

```java
public class OutboxPublisher {
    @Scheduled(fixedDelay = 500)
    public void publishPending() {
        List<OutboxEntry> entries = repo.findUnpublished(100);
        for (OutboxEntry entry : entries) {
            try {
                broker.send(entry);
                repo.markPublished(entry.id());
            } catch (Exception e) {
                log.error("publish failed for {}", entry.id(), e);
                // leave unmarked, retry on next pass
            }
        }
    }
}
```

Pros:

- Simple. Works with any DB and any broker.
- No special DB privileges needed.
- Easy to reason about and debug.

Cons:

- Latency. The event waits for the next poll. Tune the interval based on requirements.
- Polling load on the DB. The partial index on `published_at IS NULL` keeps it cheap.
- Multiple publisher instances need coordination (locking, partitioning) to avoid duplicate publishes.

### Change data capture (CDC)

A tool reads the DB's transaction log directly and emits events. Debezium is the canonical implementation for Postgres, MySQL, SQL Server, and MongoDB.

```
DB transaction -> WAL / binlog -> Debezium -> Kafka topic
```

You don't strictly need the outbox to publish a "row inserted" event. But for outbox-style structured events, you write to the outbox table, and Debezium tails it.

Pros:

- Very low latency. CDC is near-real-time.
- No application-side polling overhead.
- Captures every change, including those made outside the app.

Cons:

- More infrastructure (Debezium connector, Kafka Connect).
- Requires DB features (logical replication on Postgres, binlog on MySQL).
- Schema evolution is trickier.

Both approaches deliver at-least-once. CDC is the modern choice for high-volume systems. Polling is the right starting point for most applications.

## Consumer side: the inbox pattern

The producer side guarantees at-least-once delivery. The consumer side needs to handle duplicates.

Common pattern: an inbox table on the consumer.

```sql
CREATE TABLE inbox (
    message_id   VARCHAR(100) PRIMARY KEY,  -- broker's message ID or your own
    received_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

Consumer flow:

```
public void handle(Event event) {
    try (var tx = db.beginTransaction()) {
        if (inboxRepo.exists(event.messageId())) {
            return; // already processed, drop silently
        }
        inboxRepo.insert(event.messageId());
        applyEvent(event);
        tx.commit();
    }
}
```

The insert and the business write happen in the same transaction. If the message arrives twice, the second insert fails the unique constraint, and the consumer drops the duplicate.

This is the inbox pattern, the natural companion to the outbox.

## Order and delivery guarantees

The outbox gives at-least-once delivery. Ordering depends on the publisher and broker.

| Guarantee | How |
|---|---|
| At-least-once delivery | Publisher retries until the broker accepts the message |
| At-most-once delivery | Possible with non-durable publishing. Rarely what you want. |
| Exactly-once delivery | Doesn't exist in distributed systems. Use at-least-once + idempotent consumers. |
| Per-aggregate ordering | Publish events for the same `aggregate_id` to the same Kafka partition |
| Global ordering | Single-threaded publisher. Hurts throughput. Rarely needed. |

For most workflows, per-aggregate ordering is enough. Two different orders can be processed in any order. Events within one order must stay in sequence.

In Kafka, partition by `aggregate_id`:

```
broker.send(topic, entry.aggregateId(), entry.payload());
```

All events for the same order land in the same partition, which preserves their order.

## Cleanup strategies

The outbox table grows. Eventually you need to delete or archive old entries.

### Delete after publish

The simplest:

```
DELETE FROM outbox WHERE published_at < now() - INTERVAL '7 days';
```

The retention window (7 days here) needs to cover:

- The maximum delay before the publisher catches up.
- A debugging window. You might want to inspect events from yesterday.
- Audit retention requirements.

### Archive to cold storage

Move old rows to a separate archive table or to S3/GCS. Useful for long-term audit needs.

### Partition by time

For high-throughput outboxes, partition the table by day or week. Drop old partitions instead of deleting rows.

```
CREATE TABLE outbox (...) PARTITION BY RANGE (created_at);

CREATE TABLE outbox_2025_05 PARTITION OF outbox
    FOR VALUES FROM ('2025-05-01') TO ('2025-06-01');
```

## Implementation example

A complete Java example using Spring and Postgres.

```java
@Entity
@Table(name = "outbox")
public class OutboxEntry {
    @Id
    @GeneratedValue
    private Long id;

    private String aggregate;
    private String aggregateId;
    private String eventType;

    @Type(JsonType.class)
    @Column(columnDefinition = "jsonb")
    private Map<String, Object> payload;

    private Instant createdAt = Instant.now();
    private Instant publishedAt;
    // getters / setters
}

@Service
public class OrderService {
    private final OrderRepository orderRepo;
    private final OutboxRepository outboxRepo;

    @Transactional
    public void placeOrder(Order order) {
        orderRepo.save(order); // 1. business data

        OutboxEntry entry = new OutboxEntry();
        entry.setAggregate("Order");
        entry.setAggregateId(order.getId().toString());
        entry.setEventType("OrderPlaced");
        entry.setPayload(Map.of(
            "orderId", order.getId(),
            "customer", order.getCustomerEmail(),
            "total", order.getTotal()
        ));
        outboxRepo.save(entry); // 2. event in the same transaction
    }
}

@Component
public class OutboxPublisher {
    private final OutboxRepository repo;
    private final KafkaTemplate<String, String> kafka;
    private final ObjectMapper mapper;

    @Scheduled(fixedDelay = 500)
    public void publishPending() {
        List<OutboxEntry> batch = repo.findUnpublished(100);
        for (OutboxEntry entry : batch) {
            try {
```

```
                String json = mapper.writeValueAsString(entry.getPayload());
                kafka.send(entry.getEventType(), entry.getAggregateId(), json).get();
                entry.setPublishedAt(Instant.now());
                repo.save(entry);
            } catch (Exception e) {
                log.error("Failed to publish outbox entry {}", entry.getId(), e);
                // Leave unpublished, retry next pass
            }
        }
    }
}
```

`@Transactional` on `placeOrder` ensures both inserts commit together. The publisher confirms publish before marking entries done.

For multi-instance deployments, the publisher needs locking (Postgres advisory locks, `SELECT FOR UPDATE SKIP LOCKED`) or partition assignment so 2 publishers don't pick the same row.

## Tools

| Tool | Role |
|---|---|
| Debezium | CDC for Postgres, MySQL, SQL Server, MongoDB, and more. Feeds Kafka. |
| Kafka Connect | The framework Debezium runs on. |
| Spring Modulith | Java library with built-in outbox support for events. |
| Eventuate | Java framework for event sourcing and outbox patterns. |
| Outbox plugins | Available for many ORMs and frameworks (Hibernate, Quarkus, .NET). |

Building it by hand with a polling publisher is fine for many systems. CDC pays off when throughput is high or latency matters.

## Best practices

- One transaction for business write and outbox write. That's the whole point. If they aren't atomic, you've reinvented the dual-write problem.
- Use a monotonic ID on the outbox table. Helps preserve ordering during reads.
- Partition by aggregate ID when publishing to Kafka, so per-entity order is preserved.
- Mark, then delete. Mark entries as published when the broker confirms. Delete (or archive) later in batch.

- Bound the batch size. Don’t read 100,000 rows in one publish loop. Process in chunks.
- Add an index on unpublished rows. A partial index keeps the polling query fast as the table grows.
- Handle publisher restarts cleanly. Crashed mid-publish? The row is still unpublished. Pick it up next time. Confirm publish, then mark.
- Consumer side: implement the inbox pattern. Or use broker-level deduplication (Kafka’s transactional API).
- Monitor outbox lag. The gap between `created_at` and `published_at` tells you publishing health.
- Keep the publisher simple. No business logic. Read row, send to broker, mark published. That’s it.

## Common gotchas

- Dual-write outside the outbox. Someone adds a “while we’re here” call to an external API mid-transaction. Now the system can write to that API and then fail to commit the DB. Keep transactions DB-only.
- Publisher across multiple instances without locking. Two instances both pick up row 100, both publish, both mark. Duplicate events. Use `SELECT FOR UPDATE SKIP LOCKED` (Postgres) or partition-based assignment.
- Marking before publishing. Publisher marks the row published, then crashes before sending. The event is lost forever. Always confirm publish first, then mark.
- Unbounded outbox growth. No cleanup. Table hits millions of rows, queries slow down. Set up a retention job from day one.
- Schema drift between event and consumer. You change the event format. Old consumers can’t parse new events. Version your event schema. Use a schema registry (Confluent, Apicurio) when it gets serious.
- CDC catches every change, including ones you don’t want to publish. If you tail the table directly (not the outbox), you’ll emit events for migrations, repairs, and admin updates. The outbox table gives you control over what gets published.
- Forgetting at-least-once semantics on consumers. Without inbox deduplication, retries cause duplicate side effects. A duplicate “charge customer” event is a bad day.
- Mixing transactional and non-transactional brokers. Some brokers (Kafka with transactions) can dedupe within their boundary. Most can’t. Don’t rely on the broker for exactly-once unless you’ve configured it carefully.
- Locking the outbox table too hard. Long `SELECT FOR UPDATE` ranges block writers. Use `SKIP LOCKED` and small batches.
- Bypassing the publisher during incidents. The publisher gets stuck, someone “just publishes manually”, and the audit trail goes inconsistent. Fix the publisher instead.

## Quick reference

| Concept | Key fact |
|---|---|
| Outbox pattern | Write data and event in one DB transaction. Publish from the outbox. |
| Dual-write problem | Saving to DB and publishing to broker can't be atomic without 2PC. |
| Outbox table | Holds events to be published. Same DB as business data. |
| Polling publisher | Background job reads and publishes unpublished rows. Simple. |
| CDC | Tools like Debezium tail the DB log. Lower latency. |
| At-least-once delivery | The outbox guarantee. Consumers must be idempotent. |
| Inbox pattern | Consumer-side dedup table. Companion to outbox. |
| Partition by aggregate | Preserves per-entity ordering in Kafka. |
| Mark after confirm | Always confirm publish before marking the row done. |
| Cleanup | Delete or archive old entries. Bound the table size. |
| `SELECT FOR UPDATE SKIP LOCKED` | Postgres pattern for multi-publisher coordination. |
| Outbox lag | Gap between `created_at` and `published_at`. Monitor it. |